Workload Management

DIRAC and DiracX as compute backends

Alexandre F. Boyer · CERN

12th DIRAC(X) Users' Workshop · FZU Prague · 13–16 October 2026



The end of two decades of homogeneity

> *Sites buy from the same market as everyone else: "Hardware technology evolution now is dominated by the needs of AI" (A. Sciabà, CERN IT, July 2026).*

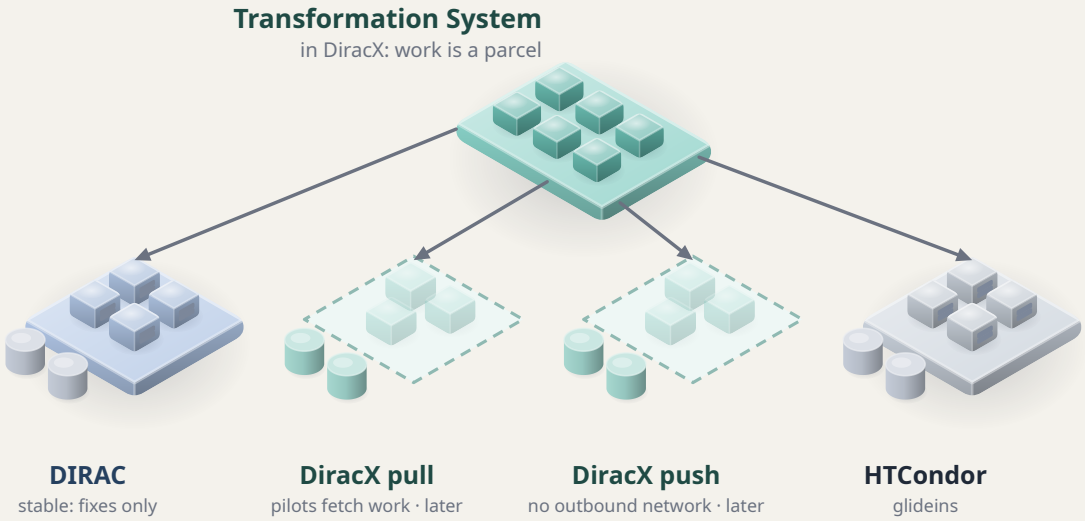
WHY IT CHANGES		WHAT SITES NOW OFFER		WHAT THE WMS MUST DO
AI expected productivity gains drive the market	→	GPUs, before they can be pledged or accounted	→	**Match on capabilities; run CPU and GPU work side by side**
Energy and hardware cost 2022 energy shock, climate targets, AI-driven prices	→	ARM next to x86: first WLCG ARM pledges in 2025	→	**One image and one match per architecture**
More cores per node clock speed stopped rising around 2005	→	Whole nodes (HPC), multicore slots (grid)	→	**Fill nodes, not single slots**
Identity X.509 gives way to the tokens industry uses	→	CEs that accept tokens, or only tokens	→	**Pilots submitted with tokens (DIRAC v8.0); DiracX built on tokens**

In DIRAC: temporary solutions, to keep it stable. **In DiracX**: the long-term solutions, designed for this.

1 · Starting from the transformation system

What it changes for workload management

The transformation system sees compute backends



each backend keeps its own data and its own data model

The transformation system (Tuesday's talks) sends parcels to compute backends. **DIRAC is one of them**, with its own data, as it is; DiracX may add a pull and a push backend later.

What separate backends give us

A stable DIRAC

The DIRAC WMS keeps its data and its behaviour. No large change: fixes and performance only, until communities move away from it.

A DiracX free of the legacy model

DiracX does not inherit DIRAC's jobs, tables or states. Work is a **parcel**; the WMS behind it is designed for it.

No rush for DiracX compute backend(s)

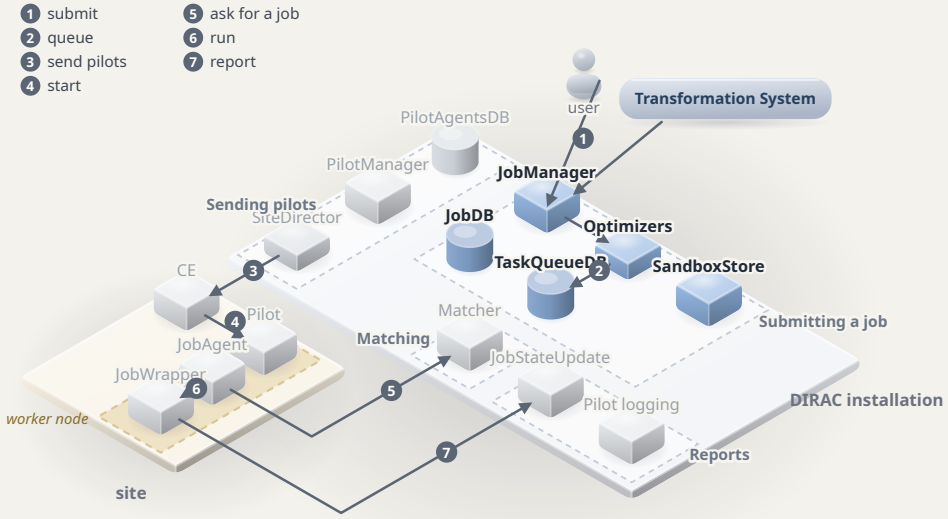
Most communities can rely on their existing WMS for now: DIRAC, or their own (see the CMS talk just before).

> The DiracX WMS is not a priority. It will move slowly, in parallel, and get a plan when the transformation system needs a native backend.

2 · DIRAC: fixes and performance

Several fixes made it faster and more reliable, for v9 and above. A walk along the path.

Submitting a job



Jobs reach their task queue 1.6× faster, and only match where they should.

FASTER

- Jobs reach their task queue at ~190 jobs/s, not ~115; one queue instead of 64 duplicates during a burst (#8801, v9.1.20).
- Missing index on job names (#8463, v9.1.0, manual `ALTER TABLE`); database objects reused by the optimizers and agents (#8457).

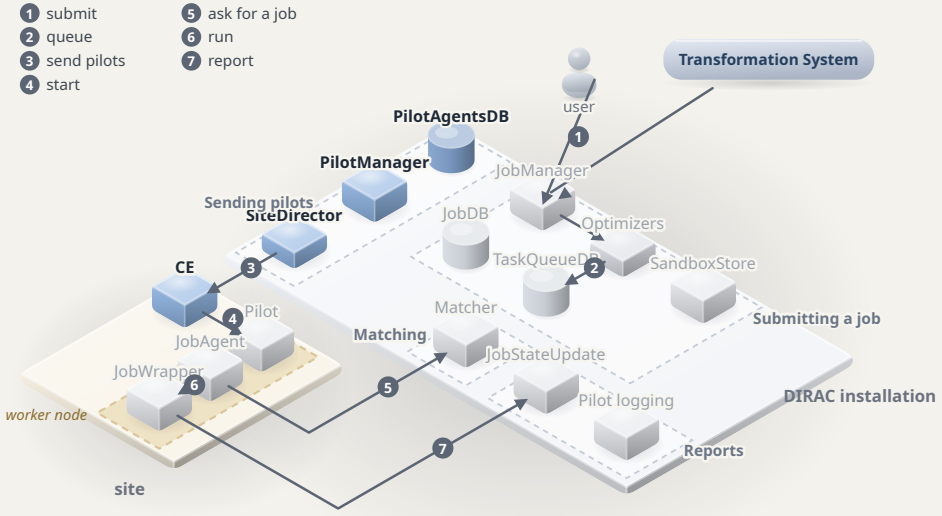
MORE RELIABLE

- A task queue could briefly lose its site restriction: a pilot at any site could then match jobs meant for other sites (#8801).

SAFER

- Sandbox store: a file name ending in `../` could write anywhere the service user can (GHSA-pwvr).
- SQL parameterised in JobDB, TaskQueueDB and the sandbox metadata (v9.1.13 to v9.1.15).

Sending pilots



Pilots only go where they can run, HPC included.

FASTER

- Indices for counting pilots per site and queue (#8463); fewer calls in the pilot agents and the PilotManager (#8569, #8624).

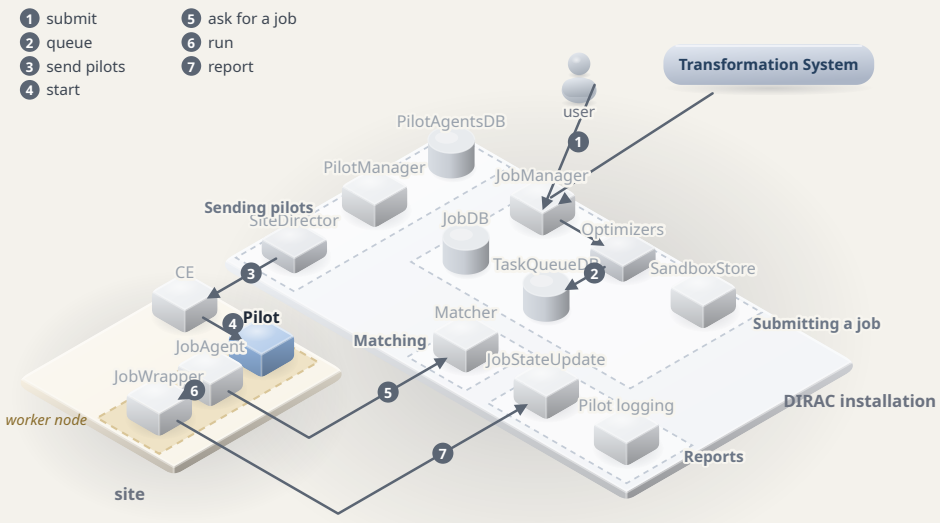
MORE RELIABLE

- The SiteDirector no longer submits to banned CEs (#8340, v9.0.2).
- HPC: the SSH computing element, rewritten on Fabric, submits through a login node to the batch system (#7703, v9.0) `DUW11`.

SAFER

- PilotManager: SQL injection and missing access checks (GHSA-7xw9, fixed in v9.1.10, v9.0.22).

The pilot on the node



Python 3 only, one source of time left, a verified download.

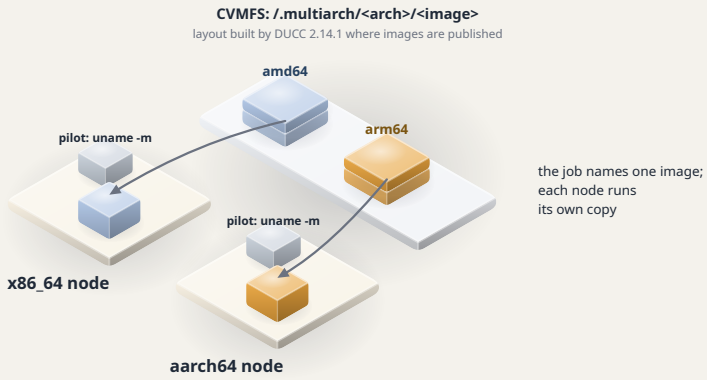
MORE RELIABLE

- Python 3 only (Pilot #268) `DUW11`.
- The pilot reads the time left once and writes it to its configuration; jobs read it there, even in containers (#8479, v9.1.4).
- One image for every architecture: next slide.

SAFER

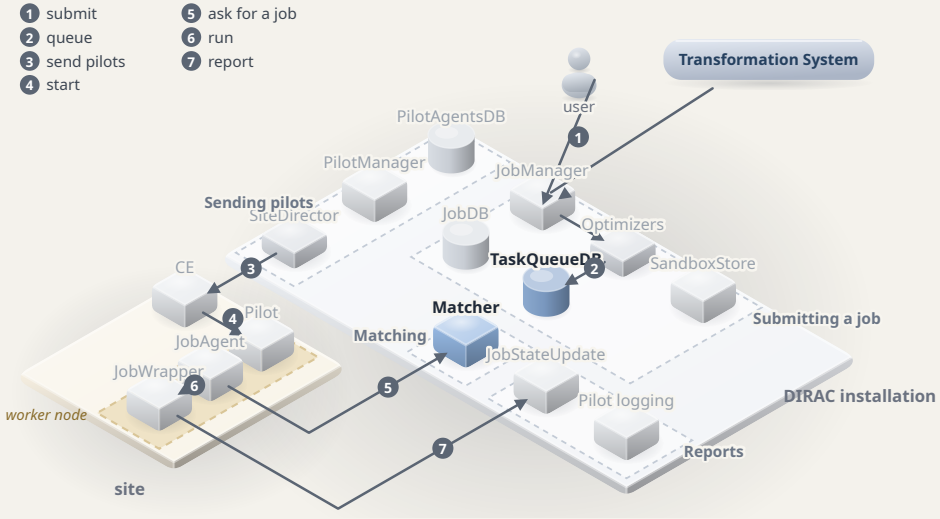
- The pilot code was fetched over HTTPS without checking the certificate, then run (GHSA-vg99, fixed in v9.1.10, v9.0.22).

One image for every architecture



- **Run each payload in the right environment**: a job names one container image, and the pilot runs the copy built for the node's architecture (x86_64 or aarch64). One of the last features added to DIRAC. #8456, open, to certify `DUW11`.
- Publishing needs CVMFS 2.14 (DUCC 2.14.1); any CVMFS client reads the result.
- **Still missing in DIRAC: alternatives.** A job states one platform and one set of requirements. It cannot say "x86_64, or aarch64, or a GPU of type A". The DiracX matcher is designed for this.

Matching

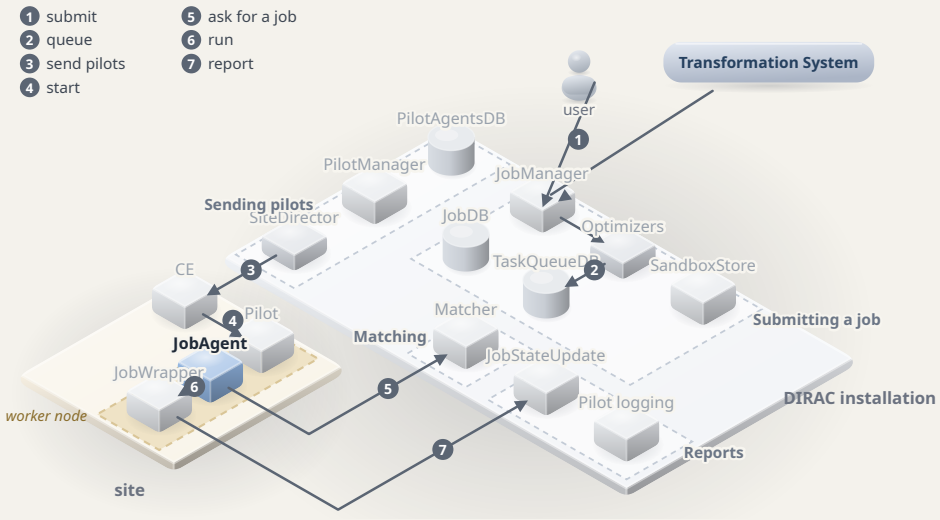


Matching up to ~17× faster.

FASTER

- Matching query: ~30 ms to ~3 ms (#8462, v9.1.0).
- When 323 task queues could serve a pilot, finding them took 3.4 s; now 0.2 s (#8801, v9.1.20).
- The delay between two matches of one job type can be below a second (#8711, v9.1.16).

The JobAgent

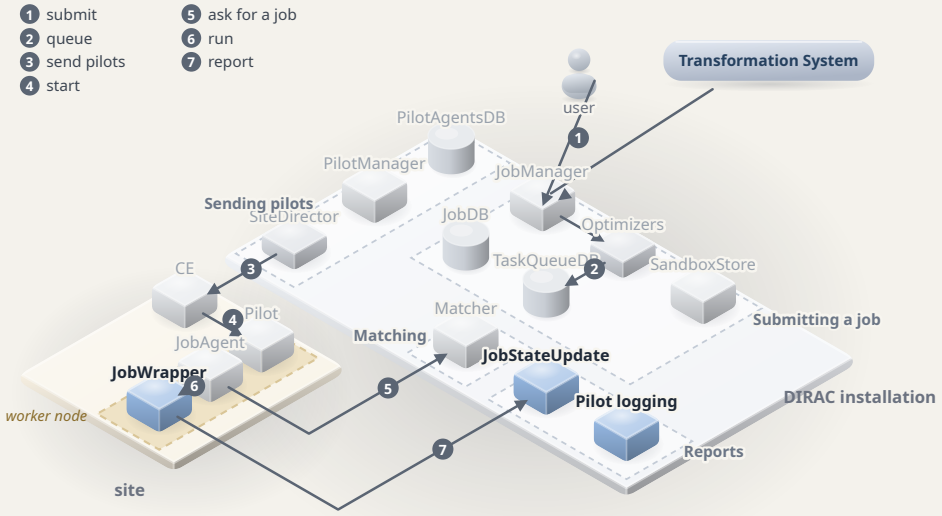


Rescheduled jobs are no longer marked Failed; each job keeps to its CPU and memory share.

MORE RELIABLE

- A job rescheduled after an early failure could already run elsewhere when the JobAgent, minutes later, marked it Failed (#8441, v9.0.17).
- Memory counts when matching and when filling a multi-core slot (#8366, #8414); cgroup v2 keeps each job within its CPU and memory share, where the site allows it (#8110) `DUW11`.
- The margin for uploads is reserved before matching (#8746, v9.1.21).

The JobWrapper and its reports



Reports DiracX accepts; nobody edits someone else's job.

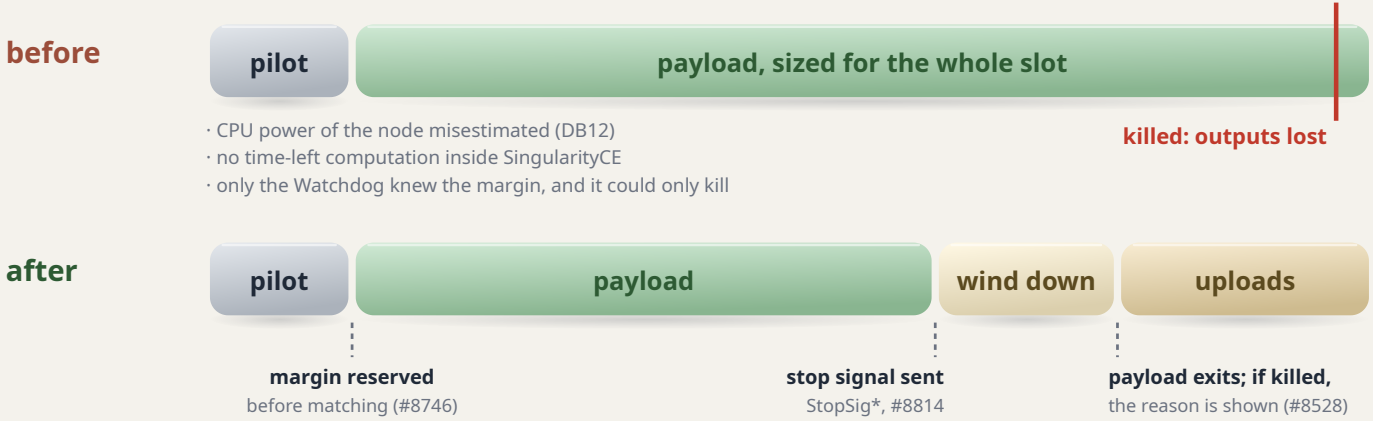
MORE RELIABLE

- Jobs shorter than the first Watchdog cycle reported NaN: invalid JSON, so DiracX rejected the whole update (#8750, v9.1.16).
- Graceful stop and the reason of a kill: next slides (#8814, #8528).

SAFER

- Job state updates: any user could change any job's owner or status (GHSA-j3xg).
- Pilot logging: a crafted file name could read any file on the server (GHSA-vm3r).
- Both fixed in v9.1.20, v9.0.26, v8.0.85.

Time left in the slot: before and after



An example: an MC job sized for 183 events produced 150, was killed, and its output was never uploaded.

One source of truth since v9.1.4 (#8479): the pilot reads the time left once, jobs read it from the pilot configuration, even inside containers.

Time left in the slot: configuration

`/Systems/WorkloadManagement/JobWrapper`, or per job in the JDL:

```
StopMargin = 300             # seconds kept for the uploads
# graceful stop, off unless set; example for Gauss
StopSigRegex = ^\S*python\S*\s+\S*/gaudirun\.py\s.*/options/Gauss/
StopSigNumber = 10           # SIGUSR1
StopSigFinishWork = 16740    # CPU work to wind down
StopSigStartWork = 8370      # minimum work before stopping
```

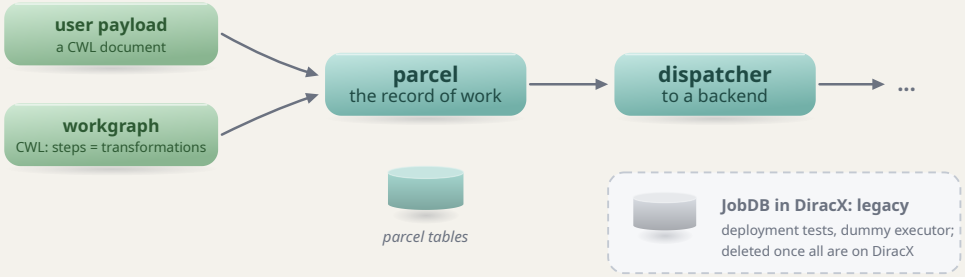
- **StopMargin** is wall clock: uploading takes the same time on any node. It is subtracted once, before matching.
- **StopSig*** thresholds are in CPU work, divided by the node's power: one value fits slow and fast nodes.
- The application must handle the signal (Gauss: `StopOnSignal`). The Watchdog still stops it when the budget is spent.

DIRAC is patched, not developed: upgrading is how you get these. Time-left fixes in **v9.1.21** only. Details: "Allocated time management" in the Administrator Guide. CPU work is normalised with DB12: next talk.

3 · DiracX: parcels, not jobs

And the proofs of concept around the future WMS

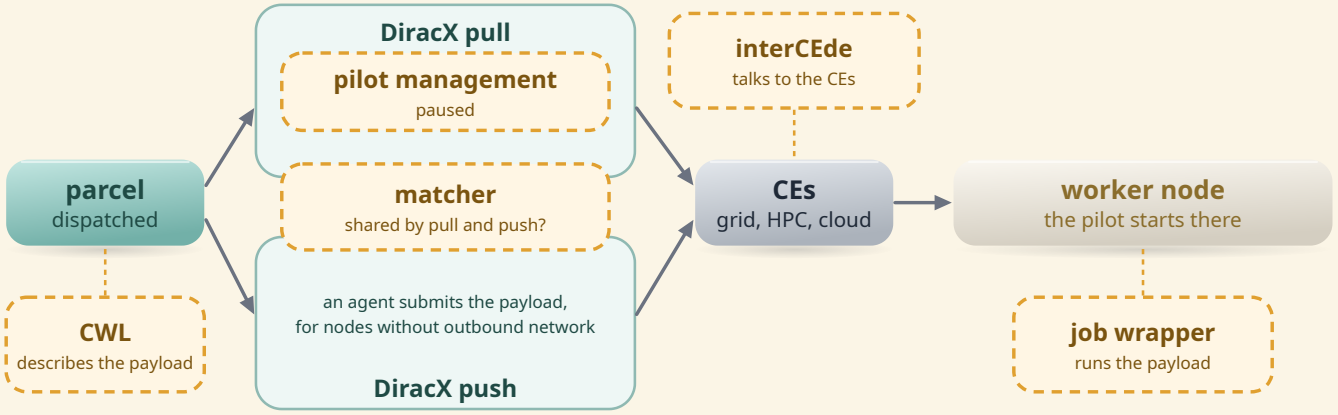
No jobs in DiracX



- A user payload is a parcel, like a bundle of transformation inputs. A failed parcel is final: resubmitting creates a new one.
- We do not port job monitoring or the job manager `DUW11`.

- The JobDB in DiracX is legacy. It receives job status updates from DIRAC, to **test that a DiracX deployment works**.
- It also feeds the **dummy job executor** (diracx#995): jobs go `Waiting → Matched → Running → Done` without running anything. A stand-in backend to test the transformation system end to end; it reads the JobDB for now, parcels later.
- **Once everyone is on DiracX, it is deleted.**

Proofs of concept around the DiracX backends



CWL	interCEde	Matcher	Pilot management	Job wrapper
ADR in draft; implemented with the transformation system	ADRs in review; core proposed; integration tests	Python baseline and load tests; Redis next; maybe shared by pull and push	for DiracX pull; ported as is, then paused	gathering requirements

The same path for each: explore, write an ADR, then implement in DiracX. Each is tested in small increments you can try.

CWL describes all the work

COMMON WORKFLOW LANGUAGE

```yaml
cwlVersion: v1.2
class: CommandLineTool
$namespaces: {dirac: https://diracgrid.org/cwl#}
hints:
  dirac:Job: {}        # a user payload (draft syntax)
baseCommand: [python, fit.py]
inputs:
  data: {type: File, inputBinding: {position: 1}}   # lfn:/... on the grid
outputs:
  plots: {type: File, outputBinding: {glob: "*.png"}}
```

WHY

One format for a user payload, a workgraph and each transformation. DiracX settings are `dirac:` hints: the same document runs on a laptop and at scale.

WHERE IT STANDS

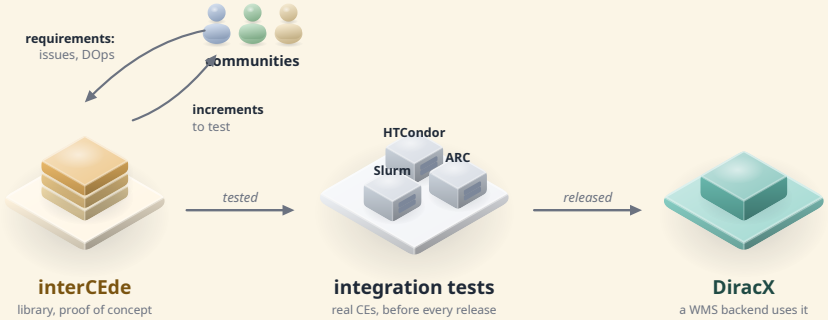
DX-ADR-007, draft. `dirac-cwl` was a first proof of concept (with CTAO): only minor details may carry over. Upstream: cwltool#2302, #2166, schema_salad#1064 merged; cwl-utils#446 in review.

NEXT

Implemented with the transformation system, from this week's hackathon.

From you: convert one of your workflows (yesterday's hands-on) and tell us what CWL cannot express.

interCEde: computing elements, explored from use cases



WHY

Talking to CEs is needed by every WMS. Explored from the use cases DIRAC covers, and those it does not but should.

WHERE IT STANDS

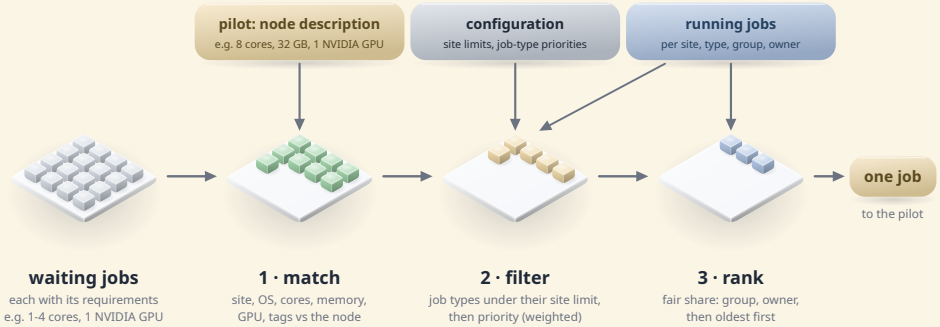
ADRs under review; core contract proposed (#27). Its own repository: integration tests against real CEs, so a change cannot break production.

NEXT

Increments you can test. The ADRs are merged once a first implementation works.

From you: if you use DIRAC computing elements, test the increments of interCEde and report what does not fit, in an issue or at a DOps.

A matcher for the DiracX WMS



one request at a time, in Python: every waiting job is checked

WHY

DIRAC's matcher will not be fast enough for what is coming (e.g. LHCb), and cannot express alternatives: GPU A, or GPU B, or CPU C.

WHERE IT STANDS

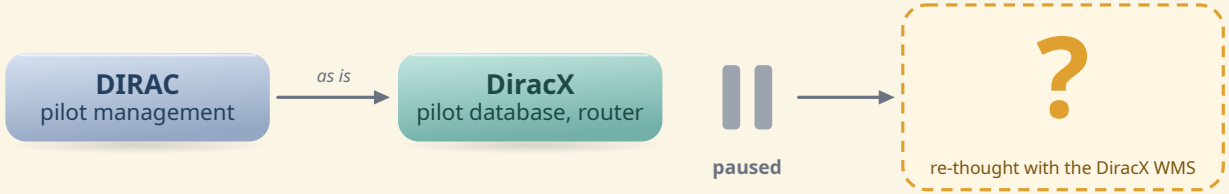
A deliberately simple Python matcher (POC), and load tests on LHCb-shaped data: 210 ms per match at 50k candidates.

NEXT

Redis, measured against this baseline, easy to rebuild (chaos tests with Redis off). Then an ADR, then DiracX.

From you: the requirements your jobs cannot express today.

Pilot management: port paused



WHY

Pilots belong to the DiracX WMS backend, which comes after the transformation system.

WHERE IT STANDS

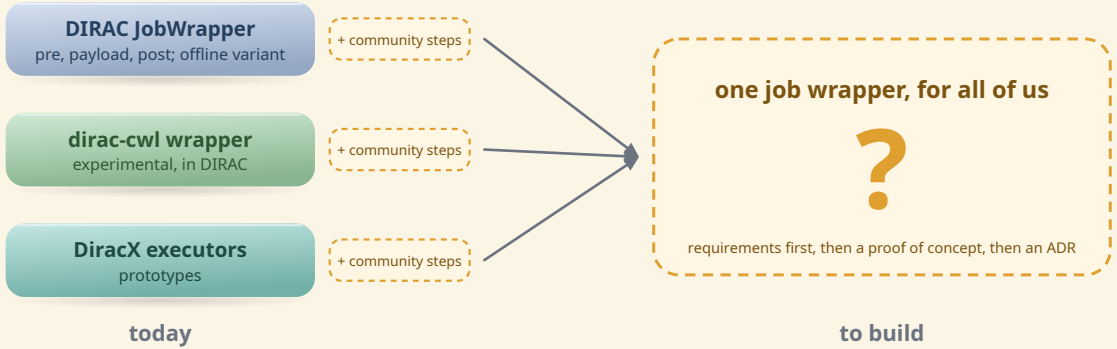
Pilot database and router ported as is (diracx#570) DUW11. The data model could be much better, and changing it would mean large changes in DIRAC: paused.

NEXT

Re-thought with the DiracX WMS backend. Meanwhile the DIRAC pilot keeps being patched.

From you: what you need from pilots: identity, logging, push or pull.

The job wrapper: one for all of us



WHY

Several wrappers today, and each community adds its own steps around the payload.

WHERE IT STANDS

Requirements to gather: inputs, time left and graceful stop, containers, nodes without outbound network, reporting, failed uploads.

NEXT

A proof of concept, then an ADR.

From you: someone to gather requirements and feedback, build the proof of concept, then design the ADR, with someone who has done it once.

Take-aways

THE MODEL

- The transformation system sends parcels to compute backends; the DIRAC WMS is one of them.
- DIRAC stays stable; DiracX is free of DIRAC's job model.

FOR INSTALLATIONS

- Upgrade to v9.1: faster matching and job insertion, time-left fixes (v9.1.21), security patches.
- Multi-architecture images: #8456, soon.

FOR DIRACX

- No jobs: parcels. The JobDB in DiracX only tests deployments, then goes.
- The DiracX WMS is not a priority; it will get a plan when needed.
- CWL describes everything that runs.
- interCEde and the matcher: exploratory, with tests; ADRs after a first implementation. Pilot management: paused.

WE NEED YOU

- Someone to lead the job wrapper requirements and proof of concept; your requirements for interCEde and the matcher.

Thank you

Questions and comments welcome, this week and after.

Alexandre F. Boyer · alexandre.boyer@cern.ch

github.com/DIRACGrid · diracx.diracgrid.org

